

目录 在去年的这个时候，我以 deepspeed-chat 的代码为例，解读了 rlhf 运作的流程。当时写这篇文章的目的，主要是想让读者在没有 RL 知识的情况下，能从直觉上快速理解这份代码，以便上手训练和修改。由于这篇文章侧重“直觉”上的解读，因此有很多描述不严谨的地方。所以去年我就想接着敲一篇 PPO 理论相关的文章，但是整理自己的笔记真得太麻烦了，所以一直 delay 到今天。所以今天这篇文章就来做这件事，我的主要参考资料是 Sutton 的这本强化学习导论。在现有的很多教材中，一般会按照这本导论的介绍方式，从 MDP（马尔可夫决策过程）和价值函数 定义介绍起，然后按照 value-based, policy-based, actor-critic 的顺序介绍。但是由于本文的重点是 actor-critic，所以我在写文章时，按照自己的思考方式重新做了整理：

我们会先介绍 policy-based 下的优化目标。

然后再介绍价值函数的相关定义。

引入 actor-critic，讨论在 policy-based 的优化目标中，对“价值”相关的部分如何做优化。

基于 actor-critic 的知识介绍 PPO。

为什么在网络上已经有无数 RL 理论知识教程的前提下，我还要再写一篇这样类型的文章呢？主要是因为：

作为一个非 RL 方向出身的人，我对 RL 的理论知识其实一直停留在“它长得是什么样”，而不是“它为什么长这样”

当我想去探究“它为什么长这样”的时候，我发现最大的难点在各类资料对 RL 公式符号定义比较混乱，或者写得太简略了。举例来说：

我们在 RL 会看到大量 $E_{x \sim p(x)}[f(x)]$ 这样求期望的形式，但是很多公式会把 E 的下标省略掉，使人搞不清楚它究竟是从哪里采样，而这点非常重要。

在 RL 的过程中，混合着随机变量和确定性变量，对于随机变量我们常讨论的是它的期望。可是在有些资料中，经常给出诸如 $r(s, a)$ 这样的形式，且不带符号说明。乍一看你很难想到它究竟代表某一次采样中确定的即时奖励，还是代表多次采样的即时奖励的期望？诸如此类

最后，这篇文章不算是 RL 理论教程，它只是我为了串起 PPO 的脉络，在自己的逻辑体系里记录的一篇笔记。如果读完这篇文章，还是有困惑的朋友，可以阅读我前面链接中给出的 Sutton 的那本教材，它更适合从 0 开始学习 RL。

【历史文章汇总】

链接

一、策略 (policy)

策略分成两种：确定性策略和随机性策略。我们用 θ 表示策略的参数。

1.1 确定性策略

$$a_t = \mu_{\theta}(s_t)$$

智能体在看到状态 s_t 的情况下，确定地执行 a_t

1.2 随机性策略

$$a_t \sim \pi_{\theta}(\cdot | s_t)$$

智能体在看到状态 s_t 的情况下，其可能执行的动作服从概率分布 $\pi_{\theta}(\cdot | s_t)$ 。也就是此时智能体是以一定概率执行某个动作 a_t 。

在我们接下来的介绍中，都假设智能体采用的是随机性策略。

二、奖励 (Reward)

奖励由当前状态、已经执行的行动和下一步的状态共同决定。

2.1 单步奖励

$$r_t = R(s_t, a_t, s_{t+1})$$

奖励和策略 π 无关

用于评估当前动作的好坏，指导智能体的动作选择。

2.2 T 步累积奖励

T 步累积奖励等于一条运动轨迹/一个回合/一个 rollout 后的单步奖励的累加。

$$R(\tau) = \sum_{t=0}^{T-1} r_t$$

2.3 折扣奖励

$$R(\tau) = \sum_{t=0}^{\infty} \gamma^t r_t$$

这里 $\gamma \in (0, 1)$ 。

三、运动轨迹 (trajectory) 和状态转移

智能体和环境做一系列/一回合交互后得到的 state、action 和 reward 的序列，所以运动轨迹也被称为 episodes 或者 rollouts，这里我们假设智能体和环境交互了 T 次：

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_{T-1}, a_{T-1}, r_{T-1})$$

s_0 是初始时智能体所处的状态，它只和环境有关。我们假设一个环境中的状态服从分布 ρ_0 ，则有 $s_0 \sim \rho_0(\cdot)$

当智能体在某个 s_t 下采取某个动作 a_t 时，它转移到某个状态 s_{t+1} 可以是确定的，也可以是随机的：

确定的状态转移： $s_{t+1} = f(s_t, a_t)$ ，表示的含义是当智能体在某个 s_t 下采取某个动作 a_t 时，环境的状态确定性地转移到 s_{t+1}

随机的状态转移： $s_{t+1} \sim P(\cdot | s_t, a_t)$

在我们接下来的介绍中，都假设环境采用的是随机状态转移。

四、Policy-based 强化学习优化目标

抽象来说，强化学习的优化过程可以总结为：

价值评估：给定一个策略 π ，如何准确评估当前策略的价值 V_{π} ？

策略迭代：给定一个当前策略的价值评估 V_{π} ，如何据此优化策略 π ？

整个优化过程由以上两点交替进行，最终收敛，得到我们想要的最优策略 π^* 和能准确评估它的价值函数 V_{π^*} 。

截自 Sutton RL 导论（对不起这个图怎么这么大啊） 此时，你肯定会想，这是否意味着强化学习过程中一定存在 π 和 V_{π} 两个实体呢？ 例如，这是否意味我们一定要训练两个神经网络，分别表示策略和价值评估？答案是否定的：

你可以只有一个价值实体 V_{π} ，因为它的输入和状态与动作相关（这里我们不区分 V 和 Q ，留到后文细说）。这意味着只要我们知道状态空间 $\mathcal{S}$ 和动作空间 $\mathcal{A}$ ， V_{π} 就可以作用到这两个空间上帮助我们衡量哪个状态/动作的价值最大，进而隐式地承担起制定策略的角色，我们也管这种方法叫 value-based。

你可以只有一个策略实体 π ，在对策略的价值评估中，我们可以让策略和环境交互多次，采样足够多的轨迹数据，用这些数据去对策略的价值做评估，然后再据此决定策略的迭代方向，我们也管这种方法叫 policy-based。

你可以同时有价值实体 V_{π} 和策略实体 π ，然后按照上面说的过程进行迭代，我们也管这种方法叫 actor-critic，其中 actor 表示策略，critic 表示价值。这是我们本文讨论的重点。

接下来，我们就直接来看 policy-based 下的强化学习优化目标：

$$\arg \max_{\pi_{\theta}} J(\pi_{\theta}) = E_{\tau \sim \pi_{\theta}}[R(\tau)] = \sum_{\tau} R(\tau) P(\tau | \pi_{\theta})$$

我们来详细解读这个目标：

τ ：表示一条轨迹序列。

π_{θ} ：智能体所采取的策略，下标 θ 表示和策略相关的参数。

$R(\tau)$ ：表示这条轨迹序列的累积奖励。

$P(\tau | \pi_{\theta})$ ：在使用策略 π_{θ} 的情况下，产出某条轨迹的概率

$\tau \sim \pi_{\theta}$ ：我们知道，当前这条轨迹序列是在使用策略 π_{θ} 的情况下采样出来的，所以 $\tau \sim \pi_{\theta}$ 隐藏的完整含义为：

$$a_{\{t\}} \sim \pi_{\{\theta\}}(a_{\{t\}} | s_{\{t\}})$$

$$s_0 \sim \rho_0(.)$$

$$s_{\{t+1\}} \sim P(.|s_{\{t\}}, a_{\{t\}})$$

$$r_{\{t\}} = R(s_{\{t\}}, a_{\{t\}}, s_{\{t+1\}})$$

$$J(\pi_{\{\theta\}}) :$$

基于策略的强化学习的总目标是，找到一个策略 $\pi_{\{\theta\}}$ ，使得它产出的轨迹的【回报期望】尽量高。回报期望表示为 $E_{\{\tau \sim \pi_{\{\theta\}}\}}[R(\tau)]$ 。

为什么这里我们讨论的是【回报期望】，而不是某一个具体的回报值？这是因为策略和状态转移具有随机性，也就是对于一个固定的策略，你让它和环境交互若干次，它每次获得的轨迹序列也是不一样的，所以 $R(\tau)$ 是个随机变量，因此我们讨论的是它的期望。从更通俗的角度来讲，你评价一个策略是否好，肯定不会只对它采样一次轨迹，你肯定需要在足够多次采样的基础上再来评估这个策略。

五、策略的梯度上升

5.1 基本推导

现在我们知道强化学习的总优化目标是：

$$\arg \max_{\pi_{\theta}} J(\pi_{\theta}) = E_{\tau \sim \pi_{\theta}}[R(\tau)] = \sum_{\tau} R(\tau) P(\tau | \pi_{\theta})$$

我们据此来计算梯度：

$$\begin{aligned} \nabla J(\pi_{\theta}) &= \sum_{\tau} R(\tau) \nabla P(\tau | \pi_{\theta}) \\ &= \sum_{\tau} R(\tau) P(\tau | \pi_{\theta}) \frac{\nabla P(\tau | \pi_{\theta})}{P(\tau | \pi_{\theta})} \\ &= \sum_{\tau} R(\tau) P(\tau | \pi_{\theta}) \nabla \log(P(\tau | \pi_{\theta})) \\ &= E_{\tau \sim \pi_{\theta}}[R(\tau) \nabla \log(P(\tau | \pi_{\theta}))] \end{aligned}$$

其中，第 2 行 ~ 第 3 行是因为：

$$\frac{d \log(f(x))}{dx} = \frac{1}{f(x)} * \frac{df(x)}{dx}$$

我们对 $\nabla \log(P(\tau | \pi_{\theta}))$ 一项再进行展开推导。

我们知道策略和状态转移都是随机的，同时我们设一条轨迹有 T 个 timestep，则我们有：

$$P(\tau | \pi_{\theta}) = \rho_0(s_0) \prod_{t=0}^{T-1} P(s_{t+1} | s_t, a_t) \pi_{\theta}(a_t | s_t)$$

据此我们继续推出：

$$\begin{aligned} \nabla \log(P(\tau | \pi_{\theta})) &= \nabla [\log \rho_0(s_0) + \sum_{t=0}^{T-1} \log P(s_{t+1} | s_t, a_t) + \sum_{t=0}^{T-1} \log \pi_{\theta}(a_t | s_t)] \\ &= \sum_{t=0}^{T-1} \nabla \log \pi_{\theta}(a_t | s_t) \end{aligned}$$

被约去的两项是因为这里我们是在对策略求梯度，而这两项和环境相关，不和策略相关。

综上，最终策略的梯度表达式为：

$$\begin{aligned} \nabla J(\pi_{\theta}) &= E_{\tau \sim \pi_{\theta}}[R(\tau) \nabla \log(P(\tau | \pi_{\theta}))] \\ &= E_{\tau \sim \pi_{\theta}}[R(\tau) \sum_{t=0}^{T-1} \nabla \log \pi_{\theta}(a_t | s_t)] \end{aligned}$$

5.2 总结

在基于策略的强化学习中，我们期望 max 以下优化目标：

$$J(\pi_{\theta}) = E_{\tau \sim \pi_{\theta}}[\sum_{t=0}^{T-1} R(\tau) \log \pi_{\theta}(a_t | s_t)]$$

基于这个优化目标，策略 π_{θ} 的梯度为：

$$\nabla J(\pi_{\theta}) = E_{\tau \sim \pi_{\theta}}[\sum_{t=0}^{T-1} R(\tau) \nabla \log \pi_{\theta}(a_t | s_t)]$$

这个梯度表达式有一个简单的直观理解：当 $R(\tau)$ 越高时，动作 a_t 贡献的梯度应该越多，这是因为此时我们认为 a_t 是一个好动作，因此我们应该提升 $\pi_{\theta}(a_t | s_t)$ ，即提升在 s_t 下执行 a_t 的概率。反之亦然。

在实践中，我们可以通过采样足够多的轨迹来估计这个期望。假设采样 N 条轨迹， N 足够大，每条轨迹涵盖 T_n 步，则上述优化目标可以再次被写成：

$$\begin{aligned} J(\pi_{\theta}) &= E_{\tau \sim \pi_{\theta}}[\sum_{t=0}^{T-1} R(\tau) \log \pi_{\theta}(a_t | s_t)] \\ &\approx \frac{1}{N} \sum_{n=0}^{N-1} \sum_{t=0}^{T_n-1} R(\tau_n) \log \pi_{\theta}(a_t | s_t) \end{aligned}$$

对应的梯度可以被写成：

$$\begin{aligned} \nabla J(\pi_{\theta}) &= E_{\tau \sim \pi_{\theta}}[\sum_{t=0}^{T-1} R(\tau) \nabla \log \pi_{\theta}(a_t | s_t)] \\ &\approx \frac{1}{N} \sum_{n=0}^{N-1} \sum_{t=0}^{T_n-1} R(\tau_n) \nabla \log \pi_{\theta}(a_t | s_t) \end{aligned}$$

六、价值函数 (Value Function)

通过上面的推导，我们知道在强化学习中，策略的梯度可以表示成：

$$\begin{aligned} \nabla J(\pi_{\theta}) &= E_{\tau \sim \pi_{\theta}}[\sum_{t=0}^{T-1} R(\tau) \nabla \log \pi_{\theta}(a_t | s_t)] \\ &\approx \frac{1}{N} \sum_{n=0}^{N-1} \sum_{t=0}^{T_n-1} R(\tau_n) \nabla \log \pi_{\theta}(a_t | s_t) \end{aligned}$$

这里 $R(\tau)$ 表示一整条轨迹的累积奖励或者累积折扣奖励。

当你端详这个公式时，你可能会有这样的疑问： $R(\tau)$ 是整条轨迹的奖励，但是 $\pi_{\theta}(a_t | s_t)$ 却是针对单步的。我用整条轨迹的回报去评估单步的价值，然后

决定要提升/降低对应 a_t 的概率，是不是不太合理呢？例如：

一条轨迹最终的回报很高，并不能代表这条轨迹中的每一个动作都是好的。

但我们又不能完全忽视轨迹的最终回报，因为我们的最终目标是让这个回合的结果是最优的。

综上，在衡量单步价值时，我们最好能在【单步回报】和【轨迹整体回报】间找到一种平衡方式。

有了以上这些直觉，你开始考虑用一个更一般的符号 Ψ_t 来表示各种可行的价值函数，你用 Ψ_t 替换掉了上面的 $R(\tau)$ ，这下策略的梯度就变成：

$$\begin{aligned} \nabla J(\pi_{\theta}) &= E_{\tau \sim \pi_{\theta}} [\sum_{t=0}^{T-1} \Psi_t \nabla \log \pi_{\theta}(a_t | s_t)] \approx \frac{1}{N} \sum_{n=0}^{N-1} \sum_{t=0}^{T[n]-1} \Psi_t \nabla \log \pi_{\theta}(a_t | s_t) \end{aligned}$$

6.1 总述：衡量价值的不同方式

总结来说 Ψ_t 可能有如下的实现方式：

- | | |
|--|---|
| 1. $\sum_{t=0}^{\infty} r_t$: total reward of the trajectory. | 4. $Q^{\pi}(s_t, a_t)$: state-action value function. |
| 2. $\sum_{t'=t}^{\infty} r_{t'}$: reward following action a_t . | 5. $A^{\pi}(s_t, a_t)$: advantage function. |
| 3. $\sum_{t'=t}^{\infty} r_{t'} - b(s_t)$: baselined version of previous formula. | 6. $r_t + V^{\pi}(s_{t+1}) - V^{\pi}(s_t)$: TD residual. |

来自 GAE paper 我们来做逐一讲解。

(1) 整条轨迹累积奖励/累积折扣奖励

这就是我们前文一直沿用的方法，即：

$$\Psi_t = R(\tau)$$

你可以通俗理解成 $R(\tau) = \sum_{t=0}^{\infty} r_t$ （省略了折扣因子）

(2) t 时刻后的累积奖励/累积折扣奖励

由于 MDP 的假设，t 时刻前发生的事情和 t 时刻没有关系，t 时刻后发生的事情才会受到 t 时刻的影响，所以我们可以令：

$$\Psi_t = \sum_{t'=t}^{\infty} r_{t'}$$

(3) 引入基线

我们沿着 (2) 继续看，假设在单次采样生成的估计中， t 时刻后的累积奖励为 $\sum_{t'=t}^{\infty} r_{t'}$ ，如果这个值很高，那一定证明在某个 s_t 下采取某个 a_t 一定好吗？答案是否定的，因为这里的“高”是一个绝对概念，而我们更想知道的是一个相对概念：这个动作究竟比别的动作好多少？同时，由于采样具有随机性，有些动作只是没被采样到，并不代表它们不好。所以这里我们引入一个基线（baseline）的方法来做调控：

$$\Psi_t = \sum_{t'=t}^{\infty} r_{t'} - b(s_t)$$

这里基线的实现方式也可以有多种，比如当我们采样了一堆轨迹，我们可以找到这些轨迹中状态为 s_t 的数据，求这些数据在 (2) 下的奖励并做平均（也就是求了个期望）当作基线。

(4) 动作价值函数 $Q_{\pi}(s_t, a_t)$

(5) 优势函数 $A_{\pi}(s_t, a_t)$

(6) 状态价值的 TD error $r_t + \gamma V_{\pi}(s_{t+1}) - V_{\pi}(s_t)$

以上三点间具有某种联系，我们这就来详细展开讲解它们。我们先关注 $Q_{\pi}(s_t, a_t)$, $V_{\pi}(s_t)$, $A_{\pi}(s_t, a_t)$ 这三者，然后再来关注 TD error。

我们沿着(4)~(6)继续来讨论 Ψ_t 的可行形式，一种符合直觉的处理方法是：

智能体来到了某个状态 s_t 下，它的动作空间是 $a_t \in \mathcal{A}$ 。智能体的策略 π 本质上是一种概率分布。它按 $\pi(a_t | s_t)$ 的概率决定要 sample 出哪个 a_t 。

而在“采样->训练->更新策略参数”的这个循环过程中，智能体要做的事情就是，如果在某个状态 s_t 下，某个动作 a_t 带来的回报“大”，那么智能体就应该提升 $\pi(a_t | s_t)$ 这个概率，也就是智能体据此不断调整 $\pi(\cdot | s_t)$ 的分布。

****那么怎么衡量在某个 s_t 下，执行某个 a_t 带来的回报是否“大”？****我们可以去计算【执行 a_t 带来的回报 - 执行其它动作的回报】，这个差值可以告诉我们 a_t 比别的动作要好多少。

那么什么叫【执行 a_t 带来的回报】和【执行其它动作带来的回报】？

假设你在玩马里奥游戏，你来到了画面的某一帧（某个 s_t ）

你在这一帧下有 3 个选择：顶金币，踩乌龟，跳过乌龟。你现在想知道执行“顶金币”的

动作比别的动作好多少。

你先执行了“顶金币”的动作（即现在你采取了某个确定的 (s_t, a_t) pair），在这之后你又玩了若干回合游戏。在每一次回合开始时，你都从（这一帧，顶金币）这个状态-动作对出发，玩到游戏结束。在每一回合中，你都记录下从（这一帧，顶金币）出发，一直到回合结束的累积奖励。你将这若干轮回合的奖励求平均，就计算出从（这一帧，顶金币）出发后的累积奖励期望，我们记其为 $Q_{\pi}(s_t, a_t)$ 。

现在你重新回到这一帧（你回到了一个确定的 s_t 上），对于“顶金币”，“踩乌龟”，“跳过乌龟”这三个动作，你按照当前的策略 $\pi(.|s_t)$ 从这三者中采样动作（注意，我们没有排除掉“顶金币”），并继续玩这个游戏直到回合结束，你记录下从 s_t 出发一直到回合结束的累积回报。重复上面这个过程若干次，然后你将这若干轮回合的奖励求平均，就计算出从（这一帧）出发后的累积奖励期望，我们记其为 $V_{\pi}(s_t)$ 。

你会发现不管是 Q 还是 V ，下标都有一个 π ，这是因为它们和你当前采取的策略是相关的

从直觉上，我们取 $A_{\pi}(s_t, a_t) = Q_{\pi}(s_t, a_t) - V_{\pi}(s_t)$ 这个差值，就可以衡量在某个状态 s_t 下，执行某个动作 a_t ，要比其它的动作好多少了。这个差值，我们可以理解为“优势” (advantage)，这个优势更合理地帮助我们衡量了单步的奖励，所以我们可以用它替换掉上面的 R_{τ} 。

当优势越大时，说明一个动作比其它动作更好，所以这时候我们要提升这个动作的概率。

通过上面的例子，我们已经引出一些关于价值函数的基本概念了：

$V_{\pi}(s_t)$ ：状态价值函数

$Q_{\pi}(s_t, a_t)$ ：动作价值函数

$A_{\pi}(s_t, a_t) = Q_{\pi}(s_t, a_t) - V_{\pi}(s_t)$ ：优势

所以接下来，我们就从理论的角度，详细展开介绍它们。

6.2 回报

在前面的例子中，我们说过，当我们从 $(s_t = \text{某一帧}, a_t = \text{顶金币})$ 出发后，我们玩游戏一直到回合结束，然后我们执行 $r_t + r_{t+1} + \dots + r_{T-1}$ ，作为这个回合的累积奖励。

但其实，我们计算这个累积奖励的目的是衡量从 (s_t =某一帧, a_t =顶金币) 这一【单步】出发后带来的未来收益。而对于这一个【单步】来说，一般离它越近的 timestep 受到它的影响越大，离它越远的 timestep 受到它的影响越小。在这个直觉的启发下，我们采用【累积折扣奖励】来定义单步（也就是某个 t 时刻）的回报：

$$G_t = r_t + \gamma r_{t+1} + \dots + \gamma^{T-t-1} r_{T-1}$$

在接下来的讲解中，提到某一个回合中【单步】的奖励，我们说的都是【累积折扣奖励】

6.3 状态价值函数 (State-Value Function)

状态价值函数的原子定义如下：

$$V_{\pi}(s_t) = E_{\pi}(G_t | s_t)$$

我们先来解释相关的符号：

首先，状态价值函数一定是和策略相关的。相同的状态 s_t 下（例如“同一帧游戏画面”），不同的策略 π 产生的结果也不一样（例如不同的人玩这个游戏）。所以我们带上了下标 π 。

其次， s_t 不是随机变量，而是一个确定值。这是因为此时我们衡量的就是从某个确定的状态 s_t 出发带来的累积奖励期望。

但是， G_t 却是一个随机变量，这是因为因为我们的策略 $\pi(\cdot | s_t)$ 和环境转移 $P(\cdot | s_t, a_t)$ 都是随机的。所以尽管每次智能体都从 s_t 出发，但采样到的轨迹却不一样。所以这里我们谈的是 G_t 的期望。

上面是状态价值函数最原子的定义，我们把这个定义展开，以便更好理解 $V_{\pi}(s_t)$ 是如何计算的：

$$\begin{aligned} V_{\pi}(s_t) &= E_{\pi}[G_t | s_t] = E_{\pi}[\sum_{t=0}^{T-1} \gamma^t r_{t+1} | s_t] = E_{\pi}[r_t + \gamma(r_{t+1} + \gamma r_{t+2} + \dots) | s_t] \\ &= E_{\pi}[r_t | s_t] + E_{\pi}[\gamma G_{t+1} | s_t] = E_{\pi}[r_t | s_t] + E_{\pi}[\gamma V_{\pi}(s_{t+1}) | s_t] \\ &= \sum_{a_t \in \mathcal{A}} \pi(a_t | s_t) \sum_{s_{t+1} \in \mathcal{S}} P(s_{t+1} | s_t, a_t) R(s_t, a_t, s_{t+1}) + \sum_{a_t \in \mathcal{A}} \pi(a_t | s_t) \sum_{s_{t+1} \in \mathcal{S}} P(s_{t+1} | s_t, a_t) \gamma V_{\pi}(s_{t+1}) \\ &= \sum_{a_t \in \mathcal{A}} \pi(a_t | s_t) \sum_{s_{t+1} \in \mathcal{S}} P(s_{t+1} | s_t, a_t) [r_t + \gamma V_{\pi}(s_{t+1})] = E_{a_t \sim \pi(\cdot | s_t)} [E_{s_{t+1} \sim P(\cdot | s_t, a_t)} [r_t + \gamma V_{\pi}(s_{t+1})]] \end{aligned}$$

$$\gamma V_{\pi}(s_{t+1}) \end{aligned}$$

上面这个展开细节帮助我们从理论上理解上面举的例子：从马里奥游戏的某一帧 s_t 出发，如何求这一帧的累积回报期望，也就是求这一帧下所有动作的累积回报期望。我们从第 4 行推导开始讲起：

第 4 ~ 第 5 行，即如何从 $E_{\pi}[\gamma G_{t+1}|s_t]$ 推到 $E_{\pi}[\gamma V_{\pi}(s_{t+1})|s_t]$ ，可以参见蘑菇书 EasyRL 的 2.2.2 的第 1 节

第 5 ~ 第 6 行讲述我们如何对期望 E 中的结果做展开。我们以 $E_{\pi}[r_t|s_t]$ 为例：

仅从这个表达式上看，它表示从某个状态 s_t 出发，在执行策略 π 的情况下， r_t 的期望。我们前面说过，由于策略和状态转移具有随机性，因此 r_t 也是个随机变量。所以这里我们讨论的是【期望】而不是某个 r_t 值。

理解了这一点，就不难理解第 6 行的关于策略和状态转移的两个求和展开项。你可以把这些求和项理解成从状态 s_t 出发，做了无数次采样后的结果。

同时， $R(s_t, a_t, s_{t+1})$ 就是我们在之前定义的奖励函数，它由三个入参决定。在我们把 $E_{\pi}[r_t|s_t]$ 展开成两个求和项后， s_t, a_t, s_{t+1} 都是确定的值而不是随机变量了，所以这里 $R(s_t, a_t, s_{t+1})$ 也是一个确定的值，从 s_t 出发某次采样过程中得到的确定值 r_t 。

第 6 行后的推导过程都比较好理解，这里不再解释。

我们在学习 rl 的过程中，会看到很多材料把上述公式写成 $V_{\pi}(s_t) = E[r_t + \gamma V_{\pi}(s_{t+1})]$ 或者 $V_{\pi}(s_t) = E[r(s_t, a_t) + \gamma V_{\pi}(s_{t+1})]$ 之类的形式，由于对 E 缺少必要的下标，这些简略的形式具有歧义性，使人混淆。所以这边我们干脆多费一点力，把所有的符号都展示出来，更方便大家深入理解 $V_{\pi}(s_t)$ 的含义。

6.4 动作价值函数 (Action-Value Function)

同样，我们先来看动作价值函数的原子定义：

$$Q_{\pi}(s_t, a_t) = E_{\pi}(G_t | s_t, a_t)$$

我们来解释相关符号：

首先，动作价值函数也是和策略 π 相关的。从之前的例子我们知道，动作价值函数衡量的是从某个确定的 (s_t, a_t) 出发后，例如从（马里奥游戏某一帧，顶金币）出发，一直到回合结束为止的累积奖励期望。在策略不一样的情况下（例如玩游戏的人不一样），即使大家都从某个确定的 (s_t, a_t) ，最后的累积奖励期望也是不一样的。所以动作价值函数一定是对某个 π 而言的。

其余要注意的地方和 $V_{\pi}(s_t)$ 一致，这里不再赘述。

$$\begin{aligned} Q_{\pi}(s_t, a_t) &= E_{\pi}[G_t | s_t, a_t] \\ &= E_{\pi}[\sum_{t=0}^{T-t} \gamma^t r_t | s_t, a_t] \\ &= E_{\pi}[r_t | s_t, a_t] + E_{\pi}[\gamma V_{\pi}(s_{t+1}) | s_t, a_t] \\ &= \sum_{s_{t+1} \in S} P(s_{t+1} | s_t, a_t) R(s_t, a_t, s_{t+1}) + \gamma \sum_{s_{t+1} \in S} P(s_{t+1} | s_t, a_t) V_{\pi}(s_{t+1}) \\ &= E_{s_{t+1} \sim P(\cdot | s_t, a_t)}[r + \gamma V_{\pi}(s_{t+1})] \end{aligned}$$

6.5 动作价值函数和状态价值函数的互相转换

我们来简单回顾下上面的内容。

状态价值函数的定义为：

$$V_{\pi}(s_t) = E_{\pi}(G_t | s_t)$$

动作价值函数的定义为：

$$Q_{\pi}(s_t, a_t) = E_{\pi}(G_t | s_t, a_t)$$

展开状态价值函数的定义，我们得到：

$$V_{\pi}(s_t) = E_{a_t \sim \pi(\cdot|s_t)}[E_{s_{t+1} \sim P(\cdot|s_t, a_t)}[r_t + \gamma V_{\pi}(s_{t+1})]]$$

展开动作价值函数的定义，我们得到：

$$Q_{\pi}(s_t, a_t) = E_{s_{t+1} \sim P(\cdot|s_t, a_t)}[r_t + \gamma V_{\pi}(s_{t+1})]$$

根据这两者的定义展开式，我们得到两者的关系为：

$$\begin{aligned} V_{\pi}(s_t) &= E_{a_t \sim \pi(\cdot|s_t, a_t)}[Q_{\pi}(s_t, a_t)] \\ &= \sum_{a_t \in \mathcal{A}} \pi(a_t|s_t) Q_{\pi}(s_t, a_t) \end{aligned}$$

关于 V 和 Q ，我们可能在脑海里一直有“ V 是 Q 的期望”这样一个模糊的印象，但是却很难做具象化的解读。希望这里通过上面马里奥游戏的例子 + 具体的推导过程，能帮助大家更深入地了解 V 和 Q 的关系。

6.6 优势函数和 TD error

在马里奥游戏的例子中，我们曾经对优势做过简单的定义：我们取 $A_{\pi}(s_t, a_t) = Q_{\pi}(s_t, a_t) - V_{\pi}(s_t)$ 这个差值，就可以衡量在某个状态 s_t 下，执行某个动作 a_t ，要比其它的动作好多少了，这个差值就是优势。

我们展开来讲优势函数，在前面的推导中我们已知：

$$Q_{\pi}(s_t, a_t) = E_{s_{t+1} \sim P(\cdot|s_t, a_t)}[r_t + \gamma V_{\pi}(s_{t+1})]$$

而对于 $V_{\pi}(s_t)$ ，我们可以把它重新改写成：

$$V_{\pi}(s_t) = E_{s_{t+1} \sim P(\cdot|s_t, a_t)}[V_{\pi}(s_t)]$$

之所以这样改写，是因为 $V_{\pi}(s_t)$ 只依赖于这个确定的 s_t ，而与 s_{t+1} 无关。

基于这两个式子，我们可以写成优势函数的表达式：

$$\begin{aligned} A_{\pi}(s_t, a_t) &= Q_{\pi}(s_t, a_t) - V_{\pi}(s_t) \\ &= E_{s_{t+1} \sim P(\cdot|s_t, a_t)}[r_t + \gamma V_{\pi}(s_{t+1})] - E_{s_{t+1} \sim P(\cdot|s_t, a_t)}[V_{\pi}(s_t)] \\ &= E_{s_{t+1} \sim P(\cdot|s_t, a_t)}[r_t + \gamma V_{\pi}(s_{t+1}) - V_{\pi}(s_t)] \\ &= E_{s_{t+1} \sim P(\cdot|s_t, a_t)}[TD_error] \end{aligned}$$

大家发现了吗：

假设这里的 V_{π} 可以准确衡量策略 π 的价值，那么 TD_error 就是优势函数的无偏估计。这意味着在期望的意义下，使用 TD_error 近似优势函数不会引起系统性的偏

差。

假设这里的 V_{π} 不能准确衡量策略 π 的价值，那么 TD_error 对于优势函数则是有偏的。这意味着由于 V_{π} 的不准确，我们无法用 $r_t + \gamma V_{\pi}(s_{t+1})$ 去近似那个真实的优势函数，因为我们将引入系统性偏差。（读到这里，你可能已经开始联想到在 actor-critic 算法下，用于估计 V_{π} 的 critic 网络在没有收敛之前都是偏离真值的，这就意味着此时我们用 TD-error 去近似优势是有偏的，所以这时我们就要请 GAE 出场了，这是后话，我们放在后文细说）

七、Actor-Critic

我们先来回顾之前定义的 policy-based 下的策略梯度：

$$\begin{aligned} \nabla J(\pi_{\theta}) &= E_{\tau \sim \pi_{\theta}} [\sum_{t=0}^{T-1} \Psi_t \nabla \log \pi_{\theta}(a_t | s_t)] \approx \frac{1}{N} \sum_{n=0}^{N-1} \sum_{t=0}^{T_n-1} \Psi_t \nabla \log \pi_{\theta}(a_t | s_t) \end{aligned}$$

其中，衡量单步价值的 Ψ_t 可以有如下几种设计方案：

基于之前的分析，我们现在选择第 6 种（TD error）作为 Ψ_t ，它衡量在某个时刻 t 选择某个动作 a 会比选择其它的动作要好多少。需要注意的是，本质上当 V_{π} 等于客观存在的真值 V_{π}^* 时，6 是 5 的无偏估计。

在 actor-critic 方法下，我们用神经网络 θ 来表示策略（actor），神经网络 ϕ 来表示价值（critic），所以这里我们进一步把 V_{π} 写成 V_{ϕ} 。注意，这可能不是一个好写法，因为 V 一定是针对某个 π 而言的，anyway 我们需要把这点记在心中，在后文的表示中可能会出现 V_{ϕ} 和 V_{π} 交替使用的场景，他们都表示一个东西，只是笔者写嗨了可能忘记从一而终了...

接下来我们来看 actor loss 和 critic loss 的具体表达式。

7.1 Actor 优化目标

现在我们可以把 actor 优化目标写成如下形式：

$$\begin{aligned} \arg \max_{\pi_{\theta}} J(\pi_{\theta}) &= E_{\tau \sim \pi_{\theta}} [\sum_{t=0}^{T-1} A_{\phi}(s_t, a_t) \log \pi_{\theta}(a_t | s_t)] \approx \frac{1}{N} \sum_{n=0}^{N-1} \sum_{t=0}^{T_n-1} (r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)) \log \pi_{\theta}(a_t | s_t) \end{aligned}$$

但是有时，一个回合中的 timesteps 可能非常多（ $T \rightarrow \infty$ ），我们无法等到回合结束再进行训练。既然我们已经使用了 TD error 来估计单步的优势，我们其实可以按照单步的方法进行更新（即你的 batch_size 是 N ，其中的每条数据都是一个单

步数据)，即 actor 优化目标可以写成：

$$\arg \max_{\pi_{\theta}} J(\pi_{\theta}) = E_{\{t\}}[A_{\phi}(s_t, a_t) \log \pi_{\theta}(a_t | s_t)]$$

在接下来的表示中，我们都将采用这种形式。

对应的策略梯度前面写过很多遍了，这里就不写出来了。

7.2 Critic 优化目标

同理，在单步更新下，我们可以把 critic 优化目标写成：

$$\arg \min_{\phi} L(\phi) = E_{\{t\}}[(r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t))^2]$$

对应的 critic 梯度这里也略去。

7.3 Actor 和 Critic 之间的关系

关于 7.1 节中，actor 这个优化目标的改写我们已经很熟悉了，但对于 7.2 中的 critic loss 你可能还是满腹疑惑，例如：看样子，critic loss 是在让优势趋于 0，但是如此一来，每个动作的好坏不就都差不多了？那你怎么能选出那个最好的动作呢？

为了解答这个问题，我们先回想第四节中提到的“价值评估->策略迭代”这样一个循环的过程：

价值评估：给定一个策略 π ，如何准确评估当前策略的价值 V_{π} ？

策略迭代：给定一个当前策略的价值评估 V_{π} ，如何据此优化策略 π

对不起，这里我们还是要再放一遍这个惊天巨图（到底怎么调图的尺寸...）我们结合 actor-critic 的框架把这个循环的优化过程展开来讲：

我们从“策略迭代步骤开始”，假设我们有一个策略 π ，且同时有一个能准确评估它的价值的 V_{π} 。基于这个 V_{π} ，我们计算某个 (s_t, a_t) 状态下动作 a_t 的优势 $A_{\pi}(s_t, v_t)$ ，如果这个优势比较高，那么我们就应该提升 $\pi(a_t | s_t)$ 这个概率。我们按照这个方式去改变策略的分布，最终会得到一个新策略 π' 。

那么，什么时候这个 π' 就是我们要找的最优策略 π^* ？我们假设当策略走到 s_t 时，有一个客观存在的最优的动作 a_t^* 。那么如果当前这个策略走到 s_t 时，产出 $\pi(a_t^* | s_t)$ 的概率已经是 1 或者是最大的，那么就证明策略已经没有提升的空间了，当前的策略已到最优（注意，这里我们始终假设 V_{π} 是能准确估计一

个策略的价值的)。

那么如果策略产出 $\pi(a_t^*|s_t)$ 的概率已经是 1 或者最大时，优势会发生什么变化？从 6.5 节中的推导里我们知道 $V_{\pi}(s_t) = \sum_{a_t \in \mathcal{A}} \pi(a_t|s_t) Q_{\pi}(s_t, a_t)$ ，由此不难得知，此时 $V_{\pi}(s_t)$ 会非常接近或者等于 $Q_{\pi}(s_t, a_t)$ ，也就是此时我们有 $A(s_t, a_t^*) = Q_{\pi}(s_t, a_t^*) - V_{\pi}(s_t) \rightarrow 0$ 。

翻译成人话来说，就是当优势趋于 0 的时候，不是说所有的动作都区分不出好坏了。而是此时策略已经趋于最优，他本来产出最优的 a_t^* 的概率就是最大的。而不是像它还没优化好之前那样，还以相当的概率产出别的 a_t 们，所以我们要在这些 a_t 们之间去比较优势。

现在我们回答“价值评估”步骤上来：此时我们做完了策略迭代，使得 $\pi \rightarrow \pi'$ ，那么之前用于衡量 π 的 V_{π} 已经不适用了，我们需要找到一个 $V_{\pi'}$ 来正确衡量 π' 。而当策略从 π 改进到 π' 后，这个 $V_{\pi}(s_t) = \sum_{a_t \in \mathcal{A}} \pi(a_t|s_t) Q_{\pi}(s_t, a_t)$ 等式右侧的分布也有所调整，所以我们应该让等式左侧去拟合等式的右侧，这样才能得出一个可以正确评估 π' 的 $V_{\pi'}$ 。

我们把以上内容换成比较好理解，但可能不太精确的人话：当我们推动 critic loss（优势）趋于 0 时：

对于 actor 来说，是推动它找到某个状态下最佳的动作，逐步向 π^* 拟合

对 critic 来说，是推动它准确衡量当前策略的价值，逐步向 V_{π^*} 拟合

八、PPO

8.1 朴素 Actor-Critic 的问题

在理解 critic loss 为何如此设计的前提下，critic 的梯度就比较好理解了，这里我们不做过多解释。我们把目光再次放回 actor 的梯度上来：

$$\begin{aligned} \nabla J(\pi_{\theta}) &= E_t[A_{\phi}(s_t, a_t)] \nabla \log \pi_{\theta}(a_t | s_t) \\ &\approx \frac{1}{N \cdot T} \sum_{n=0}^{N-1} \sum_{t=0}^{T-n-1} (r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t)) \nabla \log \pi_{\theta}(a_t | s_t) \end{aligned}$$

再次注意，这里我们写成 V_{ϕ} 的形式其实是不完整的，它只是用来刻画 ϕ 是 critic 网络的参数，我们还必须铭记 V_{ϕ} 衡量的是某个策略 π 的价值，当这个策略发生迭代而变动时， V_{ϕ} 也会变动。

观察这个梯度表达式，我们会发现如下问题：

问题 1：每次执行这个梯度更新时，我们都需要对 π_{θ} 进行若干次回合采样。我们知道智能体和环境交互的时间成本（fwd）比较高，也就是整个训练过程会比较慢。同时由于采样过程具有随机性，我们可能偶发采样到了一些方差特别大的样本，如果我们直接信任这些样本去做更新，就可能使得更新方向发生错误。

问题 2：我们在前面说过，实际训练的过程中，用 critic 网络拟合出来 V_{ϕ} 并不一定是能准确衡量 π 的那个价值函数，所以这里我们用 TD error 去估计优势其实是有偏的。为了降低这种偏差，我们需要对 $A_{\phi}(s_t, a_t)$ 进行改造，改造的方法之一就是 GAE。

接下来我们就详细来看如何解决这两个问题。

8.2 重要性采样

在朴素的方法中，我们使用 π_{θ} 和环境交互若干次，得到一批回合数据，然后我们用这个回合数据计算出来的奖励值去更新 π_{θ} 。我们管这个过程叫 on-policy（产出数据的策略和用这批数据做更新的策略是同一个）

而现在，为了降低采样成本，提升训练效率，同时更加“谨慎”地更新模型，我们想做下面这件事：

假设某次更新完毕后，我们得到策略 π_{old}

我们用 π_{old} 和环境交互，得到一批回合数据。

我们将把这一批回合数据重复使用 k 次：即我们先把这批数据喂给 π_{old} ，更新得到 π_{θ_0} ；我们再把这批数据喂给 π_{θ_0} ，更新得到 π_{θ_1} ；以此类推，做 k 次更新后，我们得到 π_{θ} 。我们管这个过程叫 off-policy（产出数据的策略和用这批数据做更新的策略不是同一个）。

在这 k 次更新后，我们令 $\pi_{\text{old}} = \pi_{\theta}$ 。重复上面的过程，直到达到设定的停止条件为止。

我们从更理论的角度来看待这个 off-policy 的过程：

假设有两个分布 $p(x)$, $q(x)$

最开始我想从 $p(x)$ 中进行多次采样，然后求函数 $f(x)$ 的期望。例如我想从 π_{θ} 中进行采样，然后求累积奖励的期望，这个期望我们表示成 $E_{x \sim p(x)}[f(x)]$

但是现在，因为某些原因，我们无法从 $p(x)$ 中直接采样，只能从另一个分布 $q(x)$ 中进行采样了，那么此时我们要怎么表示 $E_{x \sim p(x)}[f(x)]$ ？

为了解决这个问题，我们做如下变换：

$$\begin{aligned} E_{x \sim p(x)}[f(x)] &= \int p(x)f(x)dx &= \int \frac{p(x)}{q(x)}q(x)f(x)dx \\ &= E_{x \sim q(x)}[\frac{p(x)}{q(x)}f(x)] \end{aligned}$$

也就是说，当我们从不同于 $p(x)$ 的分布 $q(x)$ 上采样 x 时，从数学上我们确实有办法改写 $E_{x \sim p(x)}[f(x)]$ ，简单来说就是加上一个权重 $\frac{p(x)}{q(x)}$ ，我们管上述的转换过程叫【重要性采样】。

虽然数学上是有办法改写了，但是实际操作中，我们可能遇到 $p(x)$ 和 $q(x)$ 分布差异较大的问题。这里我直接引用李宏毅老师的课堂 ppt 来说明这一点：

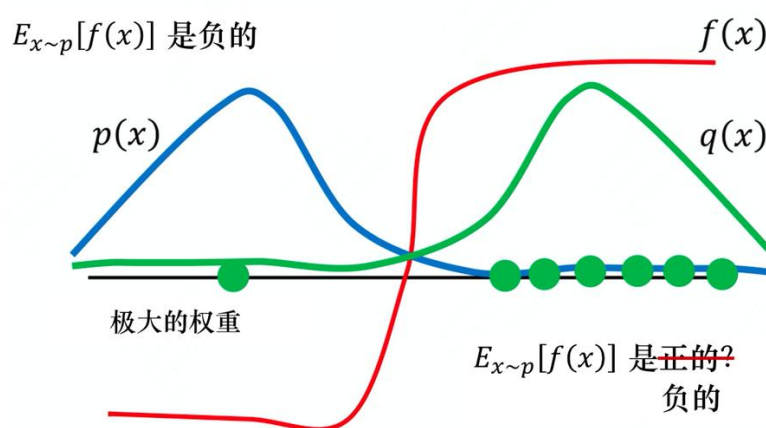


图 5.1 重要性采样的问题

我们假设 $E_{x \sim p(x)}[f(x)]$ 的真值是负数。

由于 $p(x)$ 和 $q(x)$ 差异较大。在某次采样中，我们从 $q(x)$ 里进行采样，大概率会采集到图中绿色曲线的高处，此时 $f(x)$ 是正的。也就是说，在单次采样中，我们大概率会得到一个正的 $f(x)$ 。

所以，只有经过尽可能多次的采样，让某次能命中 $q(x)$ 这个绿色曲线的低处。这时 $p(x)/q(x)$ 较大，也就赋予这个好不容易采样到的负的 $f(x)$ 非常大的权重，这才足以抵消之前正 $f(x)$ 的影响。

综上所述，当 $p(x)$ 和 $q(x)$ 差异较大时，我们需要通过足够多的采样来抵消这种差异对期望的最终影响。我们先记住这一点，在后面我们再来说这一点对我们策略的影响。

知道了重要性采样的过程，现在我们又可以根据它重写我们的优化目标了。

重要性采样前，策略的梯度是：

$$\nabla J(\pi_{\theta}) = E_t[A_{\phi}(s_t, a_t) \nabla \log \pi_{\theta}(a_t | s_t)]$$

重要性采样后，策略的梯度是：

$$\nabla J(\pi_{\theta}) = \underset{\tau \sim \pi_{\theta_{old}}}{E_t} \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} A_{\phi}(s_t, a_t) \nabla \log \pi_{\theta}(a_t | s_t) \right]$$

我们根据重要性采样构造了这个新的策略梯度，那么对应的新的 actor 优化目标就可以从这个策略梯度中反推出来：

$$\arg \max_{\pi_{\theta}} J(\pi_{\theta}) = \underset{\tau \sim \pi_{\theta_{old}}}{E_t} \left[\frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} A_{\phi}(s_t, a_t) \right]$$

特别注意 $\tau \sim \pi_{\theta_{old}}$ ，它意味着我们这一波的训练数据是由 old 策略采集来的。

但是到这步为止，我们还要在心里铭记一个问题，如何解决 π_{θ} 和 $\pi_{\theta_{old}}$ 分布差异过大的情况。我们先来看对优势函数的改进方法，然后再回来讲这个问题的解决办法。

8.3 GAE：平衡优势函数的方差和偏差

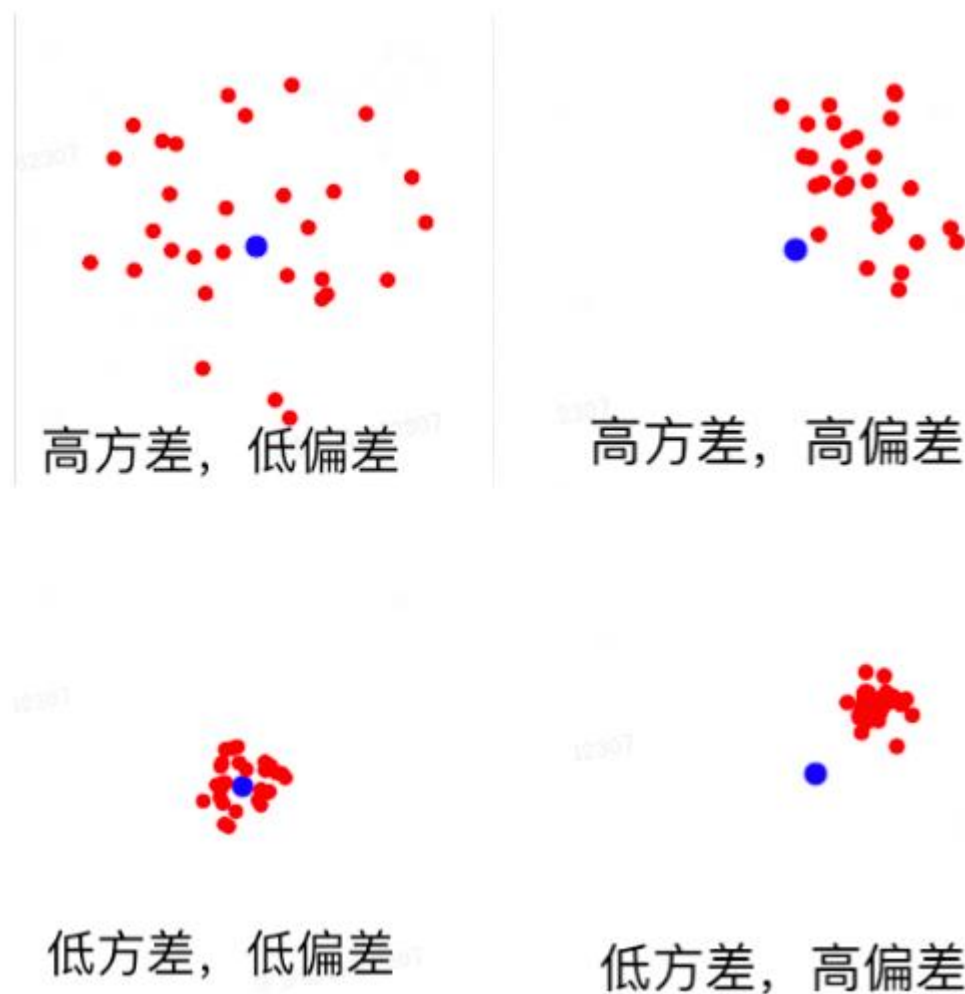
再回顾下 6.6 节的内容：在假设 V_{π} 能正确评估策略 π 的价值的前提下，我们用 TD_error 作为优势函数的无偏估计：

$$\begin{aligned} A_{\pi}(s_t, a_t) &= Q_{\pi}(s_t, a_t) - V_{\pi}(s_t) \\ &= E_{s_{t+1}} \sim$$

$$P(.|s_t, a_t))[r_t + \gamma V_{\pi}(s_{t+1})] - E_{s_{t+1} \sim P(.|s_t, a_t))}[V_{\pi}(s_t)] \quad \&= \quad E_{s_{t+1} \sim P(.|s_t, a_t))}[r_t + \gamma V_{\pi}(s_{t+1})] - V_{\pi}(s_t)] \quad \&= \quad E_{s_{t+1} \sim P(.|s_t, a_t))}[\text{TD_error}] \quad \text{\end{aligned}}$$

但是，在训练过程中，这个 V_{π} 往往无法完全正确评估出策略 π 的价值，所以上述这种估计是有偏的，也即如果我们使用 TD error 去近似优势函数，就会引发系统性偏差。这样讲可能比较抽象，我们来看具象化地解释下。

(1) 方差与偏差



低方差，低偏差： $E(\text{射击点}) = \text{靶心}$ ，且射击点密集分布在靶心周围。此时我们随机选一个射击点就能很好代表靶心

高方差，低偏差： $E(\text{射击点}) = \text{靶心}$ ，但射击点们离靶心的平均距离较远。此时随机一个射击点不能很好代表靶心，我们必须使用足够多的射击点才能估计靶心的坐标

高/低方差，高偏差： $E(\text{射击点}) \neq \text{靶心}$ ，无论你做多少次射击，你都估计不准靶心的位置。

对于优势函数我们有 $A_{\pi}(s_t, a_t) = E_{s_{t+1} \sim P(\cdot | s_t, a_t)}[r_t + \gamma V_{\pi}(s_{t+1}) - V_{\pi}(s_t)]$ ，我们可以把 $A_{\pi}(s_t, a_t)$ 当作是蓝色的靶心，而红色的点就是我们针对某个 (s_t, a_t) 做多次采样，得到的一个个 $r_t + \gamma V_{\pi}(s_{t+1}) - V_{\pi}(s_t)$ 。

当 V_{π} 能准确估计策略 π 的价值时， $r_t + \gamma V_{\pi}(s_{t+1}) - V_{\pi}(s_t)$ 至少是属于左侧“低偏差”这种情况。而对于方差它则是由这里的两个随机变量 r_t ， $V_{\pi}(s_{t+1})$ 决定的，之所以说它们是随机变量，是因为在采取某个状态对 (s_t, a_t) 的情况下，会转移到哪个 s_{t+1} 是不确定的，带来的奖励 r_t 也是不确定的。

当 V_{π} 不能准确估计策略 π 的价值时， $r_t + \gamma V_{\pi}(s_{t+1}) - V_{\pi}(s_t)$ 属于右侧“高偏差”这种情况。即红点的分布已经偏离，无论你采样多少次，你也无法估算真正的优势函数。

为了解决因为 V_{π} 估计不准而引发的“高偏差”问题，直观上我们可以尽量少信任 V_{π} 的策略，即对于 $r_t + \gamma V_{\pi}(s_{t+1}) - V_{\pi}(s_t)$ ，我们可以把 $V_{\pi}(s_{t+1})$ 做递归地展开，得到：

$$-V_{\pi}(s_t) + \sum_{l=0}^{\infty} \gamma^l r_{t+l}$$

其中， $r_t, r_{t+1}, r_{t+2}, \dots$ 都是我们某次采样得到的即时奖励数据。如果 V_{π} 不准，那么我就信任我的实际采样结果，这样至少不会让我对优势函数的估计出现偏差。

但采取这种做法又会引发一个新问题：我们知道 $r_t, r_{t+1}, r_{t+2}, \dots$ 它们都是随机变量，相比之前只用 r_t ，现在的做法带来的随机性更大了（相当于在每一个 timestep 都引入了随机性，随机性逐步累加）。也就是如果之前的方差是 $\text{Var}(r_t)$ ，那么现在的方差则变成更大的 $\text{Var}(r_t + r_{t+1} + \dots)$ 。最终，你把 $r_t + \gamma V_{\pi}(s_{t+1}) - V_{\pi}(s_t)$ 从上图的右侧纠正了上图的【高方差、低偏差】的位置。这意味着此时虽然偏差降低了，但你需要采样足够多的数据才能准确估计出优势函数，这样加重了实际训练中的采样负担。

(2) GAE

回想上面的介绍，此时你可能已自然而然想到：当 V_{π} 估计不准时，如果我能将 $r_t + \gamma V_{\pi}(s_{t+1}) - V_{\pi}(s_t)$ 和 $-V_{\pi}(s_t) + \sum_{l=0}^{\infty} \gamma^l r_{t+l}$ 间取得一种平衡就好了，即我既不想完全信任

V_{π} ，又不想完全信任我全部的采样的结果 $r_t, r_{t+1}, r_{t+2}, \dots$ ，此时，GAE (Generalized Advantage Estimator) 就登场了，它通过一个超参数 λ 控制了我们要的方差-偏差间的平衡。这里我们直接来看在 GAE 的修改下，最终的形式（具体的推导这里不再展开，大家可以看论文）：
$$\Psi_t = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l}$$

$$\delta_t = r_t + \gamma V_{\pi}(s_{t+1}) - V_{\pi}(s_t)$$

γ ：超参，折扣因子

λ ：超参，即用于平衡方差-偏差的因子。

当 λ 接近 0 时， Ψ_t 退化成 $r_t + \gamma V_{\pi}(s_{t+1}) - V_{\pi}(s_t)$ ，也即上图所绘制的高偏差情况

当 λ 接近 1 时， Ψ_t 变成 $-V_{\pi}(s_t) + \sum_{l=0}^{\infty} \gamma^l r_{t+l}$ ，也即上图所绘制的高方差-低偏差情况

综上， λ 越小，方差越小，偏差越大。 λ 越大，方差越大，偏差越小。

在接下来的表达中，我们记这种引入了 GAE 方法的单步优势为 $A_{\pi}^{\text{GAE}}(s_t, a_t)$

8.4 PPO 前身：TRPO

在引入重要性采样解决采样效率问题，引入 GAE 解决单步优势的方差-偏差平衡问题后，我们的优化目标变成： $\arg \max_{\pi_{\theta}} J(\pi_{\theta}) = \underset{\tau \sim \pi_{\theta_{old}}}{E_t} [\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} A_{\phi}^{\{GAE(s_t, a_t)\}}]$ 还记得我们在重要性采样中遗留的问题吗：如果 π_{θ} 和 π_{old} 这两个分布差异太大，且我们采样的轨迹数量没有足够大时， $J(\pi_{\theta})$ 的估计是不准确的。那要怎么办呢？一种直观的解决办法是，把 π_{θ} 和 π_{old} 的分布相似性作为 $J(\pi_{\theta})$ 的 constraint，这就是 TRPO 的做法，即我们有：

$$\begin{matrix} \arg \max_{\pi_{\theta}} J(\pi_{\theta}) = \underset{\tau \sim \pi_{\theta_{old}}}{E_t} [\frac{\pi_{\theta}(a_t|s_t)}{\pi_{old}(a_t|s_t)} A_{\phi}^{\{GAE(s_t, a_t)\}}] \\ \text{subject to ; } E_t[KL(\pi_{\theta_{old}}(.|s_t), \pi_{\theta}(.|s_t))] \leq \delta \end{matrix}$$
 由于这种限制不直接加在 $J(\pi_{\theta})$ 中，因此使得整体优化过程变得较为复杂，这就是 TRPO 的缺陷（关于 TRPO 的更多细节，这里就不做展开了，大家可以去看 paper）

8.5 PPO 做法 1：PPO-Clip

当我们把 π_{θ} 和 π_{old} 的分布相似性作为 $J(\pi_{\theta})$ 的 constraint 时，整个优化过程是复杂的，所以 PPO 在这里做的事情是：我能不能把这个限制放回 $J(\pi_{\theta})$ 中？而 PPO-Clip 就是其中一种放回办法。我们重新来端详引入重要性采样和 GAE 以后的强化学习优化目标：

$$\arg \max_{\pi_{\theta}} J(\pi_{\theta}) = \underset{\tau \sim \pi_{\theta_{old}}}{E_t} [\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} A_{\phi}^{\{GAE(s_t, a_t)\}}]$$

当 $A_{\phi}^{\{GAE(s_t, a_t)\}} > 0$ 时，说明我们当前执行的动作 a_t 相较于别的动作更好，因此我们要提升 $\pi_{\theta}(a_t|s_t)$ 。但是考虑到在采样不足的情况下， π_{θ} 和 $\pi_{\theta_{old}}$ 的分布差异不能太大，因此 $\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ 是有上限的，即不能一味轻信 $A_{\phi}^{\{GAE(s_t, a_t)\}}$ 而持续不断提升 $\pi_{\theta}(a_t|s_t)$ ，你要保证 π_{θ} 在 $\pi_{\theta_{old}}$ 的信任阙内。我们假设当这个比值 $\geq 1 + \epsilon$ 的时候，就对这个比值做 clip，固定在 $1 + \epsilon$ 。

当 $A_{\phi}^{\{GAE(s_t, a_t)\}} < 0$ 时，说明我们当前执行的动作 a_t 相较于别的动作更差，因此我们要降低 $\pi_{\theta}(a_t|s_t)$ 。考虑到 π_{θ} 和 π_{old} 的分布差异不能太大，因此 $\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ 是有下限的，即不能一味轻信 $A_{\phi}^{\{GAE(s_t, a_t)\}}$ 而持续不断降低 $\pi_{\theta}(a_t|s_t)$ ，我

们假设当这个比值 $\leq 1 - \epsilon$ 的时候，就对这个比值做 clip，固定在 $1 - \epsilon$ 。

ϵ 就是 PPO 其中的一个超参。

基于这种 clip 思想，我们得出了 ppo-clip 下强化学习的优化目标，为了表达简便，我们这里令 $r_t(\theta) = \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ ：

$$J^{CLIP}(\pi_{\theta}) = \underset{\tau \sim \pi_{\theta_{old}}}{E_t} [\min(r_t(\theta) A_{\phi}^{GAE}(s_t, a_t), \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) A_{\phi}^{GAE}(s_t, a_t))]$$

虽然这个公式看起来复杂，但本质上做的就是上面说的 clip 问题，大家对照着上面的解释看这个公式就好。

8.6 PPO 做法 2: PPO-Penalty

除了 PPO-Clip 的方法外，我们还可以采用 PPO-Penalty 的方法来解决 TRPO 优化复杂的问题。PPO-Penalty 做的事情就更直观了，直接把限制条件放进优化目标中，而这个限制条件就被称为“KL penalty”，PPO-Penalty 的优化目标如下：

$$\max_{\pi_{\theta}} J(\pi_{\theta}) = \underset{\tau \sim \pi_{\theta_{old}}}{E_t} [\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)} A_{\phi}^{GAE}(s_t, a_t) - \beta \text{KL}(\pi_{\theta_{old}}(.|s_t), \pi_{\theta}(.|s_t))] \text{ 其中，超参 } \beta \text{ 的调整策略如下：}$$

首先，我们对 KL 散度一项也会设置 threshold，我们分别记为 $\text{KL}_{\max}$, $\text{KL}_{\min}$

当 $\text{KL} \geq \text{KL}_{\max}$ 时，说明当前策略已经偏离 old 策略较远了，这时我们应该增大 β ，把分布拉回来。

当 $\text{KL} \leq \text{KL}_{\min}$ 时，说明当前策略很可能找到了一条捷径，即它只优化 KL 散度一项，让自己和 old 更相近，而不去优化前面优势相关的项，所以这时我们应该减小 β ，降低 KL 散度一项的影响

(PPO 论文里的表述和这里又一些不一样，但本质上都是用 min 和 max 去动态变更 β)

8.7 PPO 中的 critic loss

在 PPO 的原始论文中，并没有对 critic 和 actor 拆分成两个网络以后的 critic loss 形式做详细介绍，所以这部分的解读我直接以 deepspeed-chat 的 rlhf 实现为例，讲一下

critic loss 的实现。 我们知道，PPO 的更新步骤是：

对于每一个 batch 的数据

for i in steps:

 # 先收集经验值

 exps = generate_experience(prompts, actor, critic, reward, ref)

 # 一个 batch 的经验值将被用于计算 ppo_epochs 次 loss，更新 ppo_epochs 次模型

 # 这也意味着，当你计算一次新 loss 时，你用的是更新后的模型

 for j in ppo_epochs:

 actor_loss = cal_actor_loss(exps, actor)

 critic_loss = cal_critic_loss(exps, critic)

 actor.backward(actor_loss)

 actor.step()

 critc.backward(critic_loss)

 critic.step()

在以上的讲解中，我们已经知道 actor 是在 PPO epoch 中使用同一批数据做迭代更新的。但是同时，critic 也是在迭代更新的。为什么 critic 要做迭代更新？因为 V_{π} 是和 π 挂钩的，如果你的策略参数已经更新了，你必须有一个新的 critic 来衡量新策略的价值。因此，ppo 中的 critic loss 被设计为（这里我们写的是单步 Value，所以我们再做一次符号的简写，把下标 ϕ 变成 t ，但是 critic 网络依然是由参数 ϕ 定义的）：

$$V_t^{\text{CLIP}} = \text{clip}(V_t^{\text{new}}, V_t^{\text{old}} - \epsilon, V_t^{\text{old}} + \epsilon)$$
$$A_t^{\text{GAE}} = V_t^{\text{old}} - \arg \min_{\phi} L(V_{\phi}) = E_t[\max(V_t^{\text{new}} - R_t)^2, (V_t^{\text{CLIP}} - R_t)^2]$$
 我们来详细解释这个公式的含义：

首先，我们先根据 8.3 (2) 中对 GAE 的定义，递归地展开 A_t^{GAE} ，得到：

$$A_t^{\text{GAE}} = \delta_t + \gamma \lambda A_{t+1}^{\text{GAE}} = (r_t + \gamma * V_{t+1}^{\text{old}} - V_t^{\text{old}}) + \gamma \lambda A_{t+1}^{\text{GAE}}$$

基于这个表达式，本质上 $R_t = A_t^{\text{GAE}} + V_t^{\text{old}} = (r_t + \gamma * V_{t+1}^{\text{old}}) + \gamma \lambda A_{t+1}^{\text{GAE}}$

在 ppo epoch 开始前，我们有初始 actor 和初始 critic。其中初始 critic 就是上面所说的 V_t^{old}

我们用初始 actor 和初始 critic 产生经验值，然后这波经验值将被用在接下来 ppo epochs 轮的迭代中，也就是在这些迭代中，上式里的 $r_t, V_t^{\text{old}}, V_{t+1}^{\text{old}}, A_t^{\text{GAE}}, A_{t+1}^{\text{GAE}}, R_t$ 都来自这些初始的经验值，它们在接下来的这个 ppo epochs 轮迭代中是不变的。

本质上，critic 的优化目标是以下差值的平方： $V_t^{\text{new}} - R_t = [V_t^{\text{new}} - (r_t + \gamma * V_{t+1}^{\text{old}})] - \gamma \lambda A_{t+1}^{\text{GAE}}$ 。这里我们先不考虑 clip，因为 clip 只是一种 trick。

大家发现了吗，这个 critic 的优化目标就和我们 7.2 节中说的 critic 优化目标是一致的，只不过这里引入了 GAE 对优势做了指数平滑而已。

那么为什么 critic 要选择用这个优化目标？答案请看 7.3 节中的解释。

在理解了这一些基本点的基础上，我们再来考虑引入 clip 的作用，它和 actor 的 ppo-clip 一样，本质上是想让 critic 在信任阈内更新：

当 $V_t^{\text{new}} < R_t$ 时，我们确实应该鼓励 V_t^{new} 向 R_t 提升，但是如果它本身的值已经超出 V_t^{old} 的信任阈 $V_t^{\text{old}} + \epsilon$ 之外，此时我们就应该对它做 clip。也就是说，如果此时我们应用 max，最终取的是 $(V_t^{\text{CLIP}} - R_t)^2$ ，注意， V_t^{CLIP} 最终是由 V_t^{old} 定义的，而这一项来自我们之前收集的经验，是个独立的常数，它和等待更新的这个 critic 无关，所以相当于我们本次不对 critic 做梯度更新。

当 $V_t^{\text{new}} > R_t$ 时，也是同理类推，这里不赘述。

在本文中，我们从理论上介绍了策略梯度->Actor-Critic->PPO 的理论框架，在下一篇中，我们将会重新结合 rlhf 的代码，从更加严谨的角度解读这份代码。

九、参考

- 1、Reinforcement Learning: An Introduction
- 2、蘑菇书 EasyRL
- 3、各类 paper，这里不一一例举